

PROCESSOS ESTOCÁSTICOS II

Trabalho Cadeias de Markov

Mateus Ribeiro

Apr 14 2026

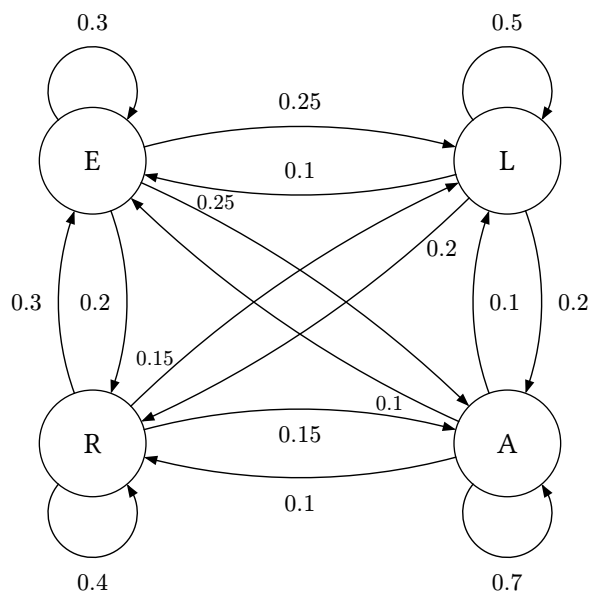
Prof. Charles Casimiro

- Obs: A resolução está dividida entre esse documento onde se encontram as questões teóricas e os resultados das computacionais e um jupyternotebook com os códigos necessários, além disso os códigos também estão listados no final do pdf.

Questão 1

Mostre a representação gráfica dos estados da cadeia de Markov e suas transições possíveis

Dada a matriz de probabilidade de transição de um passo do sistema, temos que sua representação gráfica é dada por:



Questão 2

Faça um código, em qualquer linguagem da sua escolha, que faça as seguintes avaliações sobre o processo modelado pela cadeia de Markov descrito, assumindo a ordem dos estados como E,R,L,A.

- i. Mostre o valor da matriz de transição de n passos, $P(n)$, para $n = 1, 2, 5, 10, 20, 50, 100$.
 - ii. Mostre que a matriz de probabilidades de transição aponta para convergência ao longo das iterações.
 - iii. Para $n = 1, 2, 5, 10, 20, 50, 100$, calcular a probabilidade de estar em cada estado após n passos, começando de cada estado inicial arbitrário (de sua escolha).
 - iv. Qual estado tem maior probabilidade estacionária? Ilustre os valores do vetor de probabilidades estacionárias.
 - v. Se o sistema for iniciado com as seguintes probabilidades iniciais dos estados $p(0) = [0.3, 0.2, 0.1, 0.4]$ (novamente assumindo a ordem E, R, L, A), verifique quão rápido o sistema “esquece o estado inicial”? Ou seja, a partir de que momento o sistema aproxima das probabilidades estacionárias? Mostre os gráficos que lhe permitiram concluir esse ponto.
- i. Sabemos que, de acordo com a equação de Chapman-Kolmogorov, conseguimos obter a matriz de transição para um número genérico de $n + m$ passos através da seguinte relação:

$$p_{ij}(m+n) = \sum_k p_{ik}(m) \cdot p_{kj}(n)$$

De acordo com a equação acima, temos que:

$$\begin{aligned} P(n+m) &= \{p_{ij}(n+m)\} \\ \Rightarrow P(n+m) &= P(n) \cdot P(m) \\ \Rightarrow P(n) &= P^n = \prod_{i=1}^n P \end{aligned}$$

onde P é a matriz de transição de um passo.

Com isso, temos que:

$$P^1 = \begin{pmatrix} 0.3 & 0.2 & 0.25 & 0.25 \\ 0.3 & 0.4 & 0.15 & 0.15 \\ 0.1 & 0.2 & 0.5 & 0.2 \\ 0.1 & 0.1 & 0.1 & 0.7 \end{pmatrix}$$

$$P^2 = \begin{pmatrix} 0.2 & 0.215 & 0.255 & 0.33 \\ 0.24 & 0.265 & 0.225 & 0.27 \\ 0.16 & 0.22 & 0.325 & 0.295 \\ 0.14 & 0.15 & 0.16 & 0.55 \end{pmatrix} \quad P^5 = \begin{pmatrix} 0.177 & 0.203 & 0.231 & 0.389 \\ 0.179 & 0.206 & 0.233 & 0.381 \\ 0.177 & 0.205 & 0.234 & 0.384 \\ 0.171 & 0.194 & 0.218 & 0.417 \end{pmatrix}$$

$$P^{10} = \begin{pmatrix} 0.175 & 0.2 & 0.227 & 0.397 \\ 0.175 & 0.2 & 0.227 & 0.397 \\ 0.175 & 0.2 & 0.227 & 0.397 \\ 0.175 & 0.2 & 0.227 & 0.398 \end{pmatrix}$$

$$P^{20} = \begin{pmatrix} 0.175 & 0.2 & 0.227 & 0.397 \\ 0.175 & 0.2 & 0.227 & 0.397 \\ 0.175 & 0.2 & 0.227 & 0.397 \\ 0.175 & 0.2 & 0.227 & 0.397 \end{pmatrix}$$

$$P^{50} = \begin{pmatrix} 0.175 & 0.2 & 0.227 & 0.397 \\ 0.175 & 0.2 & 0.227 & 0.397 \\ 0.175 & 0.2 & 0.227 & 0.397 \\ 0.175 & 0.2 & 0.227 & 0.397 \end{pmatrix}$$

$$P^{100} = \begin{pmatrix} 0.175 & 0.2 & 0.227 & 0.397 \\ 0.175 & 0.2 & 0.227 & 0.397 \\ 0.175 & 0.2 & 0.227 & 0.397 \\ 0.175 & 0.2 & 0.227 & 0.397 \end{pmatrix}$$

- ii. Para demonstrar a convergência da matriz de transição ao longo das iterações, calcularemos tanto a diferença entre as matrizes que se sucedem (os resultados estão no jupyter notebook) quanto a soma dos quadrados das diferenças entre os elementos das matrizes nos passos sucessivos da nossa amostra (por exemplo, a diferença entre a matriz no passo 100 e a do passo 50).

Ex.:

$$\sum_i \sum_j (p_{ij}(100) - p_{ij}(50))^2$$

Ao plotar esses valores para os passos sucessivos, podemos observar que a soma dos quadrados das diferenças converge para 0.

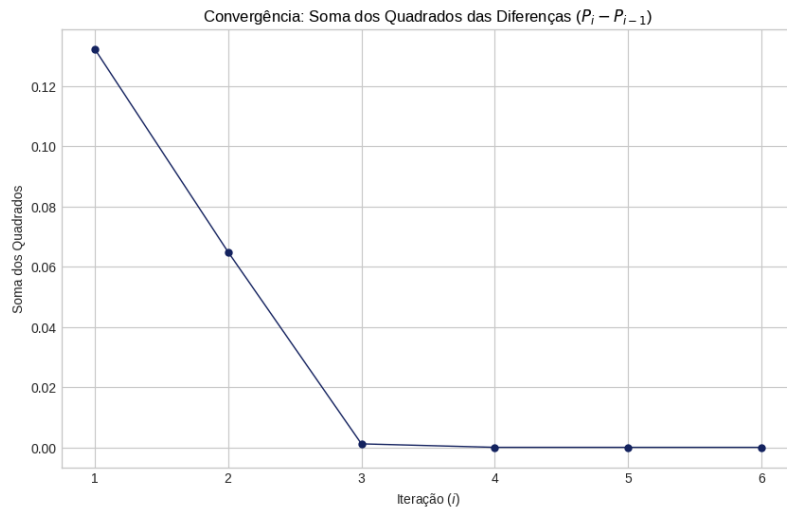


Figure 1: Convergence plot

- iii. Para avaliar como o sistema se comporta após n passos para $n = 1, 2, 5, 10, 20, 50, 100$ começando de estados arbitrários (escolha a meu critério) será assumido 4 cenários possíveis:

- Começando com 100% de certeza no estado “E”
- Começando com 100% de certeza no estado “R”
- Começando com 100% de certeza no estado “L”
- Começando com 100% de certeza no estado “A”

Para cada um desses cenários, temos que o vetor de probabilidades após n passos é dado pela seguinte relação:

$$p(n) = p(0)P(n) = p(0)P^n \quad (1)$$

Vemos que, analisando os distintos cenários, as probabilidades resultantes a curto prazo são bem diferentes entre si mas convergem para os mesmos valores ao longo dos passos.

Passo (n)	$p_E(n)$	$p_R(n)$	$p_L(n)$	$p_A(n)$
Começando 100% no Estado E ($p(0) = [1, 0, 0, 0]$)				
0	1.0000	0.0000	0.0000	0.0000
1	0.3000	0.2000	0.2500	0.2500
2	0.2000	0.2150	0.2550	0.3300
5	0.1768	0.2030	0.2311	0.3890
10	0.1751	0.2004	0.2273	0.3972
20	0.1751	0.2003	0.2271	0.3975
50	0.1751	0.2003	0.2271	0.3975
100	0.1751	0.2003	0.2271	0.3975
Começando 100% no Estado R ($p(0) = [0, 1, 0, 0]$)				
0	0.0000	1.0000	0.0000	0.0000
1	0.3000	0.4000	0.1500	0.1500
2	0.2400	0.2650	0.2250	0.2700
5	0.1794	0.2058	0.2333	0.3815
10	0.1752	0.2005	0.2274	0.3970
20	0.1751	0.2003	0.2271	0.3975
50	0.1751	0.2003	0.2271	0.3975
100	0.1751	0.2003	0.2271	0.3975
Começando 100% no Estado L ($p(0) = [0, 0, 1, 0]$)				
0	0.0000	0.0000	1.0000	0.0000
1	0.1000	0.2000	0.5000	0.2000
2	0.1600	0.2200	0.3250	0.2950
5	0.1773	0.2047	0.2344	0.3836
10	0.1752	0.2005	0.2273	0.3970
20	0.1751	0.2003	0.2271	0.3975
50	0.1751	0.2003	0.2271	0.3975
100	0.1751	0.2003	0.2271	0.3975
Começando 100% no Estado A ($p(0) = [0, 0, 0, 1]$)				
0	0.0000	0.0000	0.0000	1.0000
1	0.1000	0.1000	0.1000	0.7000
2	0.1400	0.1500	0.1600	0.5500
5	0.1708	0.1939	0.2181	0.4172
10	0.1749	0.2001	0.2268	0.3981
20	0.1751	0.2003	0.2271	0.3975
50	0.1751	0.2003	0.2271	0.3975
100	0.1751	0.2003	0.2271	0.3975

Table 1: Evolução e convergência do vetor de probabilidade para diferentes estados iniciais.

- iv. Para calcular o vetor de probabilidades estacionárias temos que resolver o seguinte sistema de equações (dado em notação matricial):

$$\pi \cdot P = \pi$$

Onde:

- π é o vetor de probabilidades estacionárias, ou seja, o vetor que queremos encontrar.
- P é a matriz de transição de um passo.

Ou seja,

$$(\pi_E \quad \pi_R \quad \pi_L \quad \pi_A) \cdot \begin{pmatrix} 0.3 & 0.2 & 0.25 & 0.25 \\ 0.3 & 0.4 & 0.15 & 0.15 \\ 0.1 & 0.2 & 0.5 & 0.2 \\ 0.1 & 0.1 & 0.1 & 0.7 \end{pmatrix} = (\pi_E \quad \pi_R \quad \pi_L \quad \pi_A)$$

Além disso, por se tratarem de probabilidades, sabemos que $\pi_E + \pi_R + \pi_L + \pi_A = 1$

Com isso, nós temos o seguinte sistema de 5 equações:

$$\begin{cases} -0.7\pi_E + 0.3\pi_R + 0.1\pi_L + 0.1\pi_A = 0 & (1) \\ 0.2\pi_E - 0.6\pi_R + 0.2\pi_L + 0.1\pi_A = 0 & (2) \\ 0.25\pi_E + 0.15\pi_R - 0.5\pi_L + 0.1\pi_A = 0 & (3) \\ 0.25\pi_E + 0.15\pi_R + 0.2\pi_L - 0.3\pi_A = 0 & (4) \\ \pi_E + \pi_R + \pi_L + \pi_A = 1 & (5) \end{cases}$$

Resolvendo o sistema

O objetivo é encontrar os valores de todos os estados em função de π_L para, no final, descobrir seu valor numérico usando uma equação independente.

Passo 1: Encontrar π_A em função de π_L

Subtraindo a Equação (4) da Equação (3), eliminamos π_E e π_R :

$$\begin{aligned} (0.25\pi_E + 0.15\pi_R + 0.2\pi_L - 0.3\pi_A) - (0.25\pi_E + 0.15\pi_R - 0.5\pi_L + 0.1\pi_A) &= 0 \\ 0.7\pi_L - 0.4\pi_A &= 0 \\ 0.4\pi_A &= 0.7\pi_L \\ \pi_A &= 1.75\pi_L \end{aligned}$$

Passo 2: Encontrar π_E em função de π_L e π_R

Utilizando a equação da soma das probabilidades (Equação 5) e substituindo o valor encontrado para π_A :

$$\begin{aligned} \pi_E + \pi_R + \pi_L + 1.75\pi_L &= 1 \\ \pi_E + \pi_R + 2.75\pi_L &= 1 \\ \pi_E &= 1 - \pi_R - 2.75\pi_L \end{aligned}$$

Passo 3: Encontrar π_R em função de π_L

Substituímos os valores de π_A e π_E encontrados anteriormente na Equação (1):

$$\begin{aligned} -0.7(1 - \pi_R - 2.75\pi_L) + 0.3\pi_R + 0.1\pi_L + 0.1(1.75\pi_L) &= 0 \\ -0.7 + 0.7\pi_R + 1.925\pi_L + 0.3\pi_R + 0.1\pi_L + 0.175\pi_L &= 0 \end{aligned}$$

$$-0.7 + 1.0\pi_R + 2.2\pi_L = 0$$

$$\pi_R = 0.7 - 2.2\pi_L$$

Obs: Com isso, podemos atualizar π_E para ficar apenas em função de π_L :

$$\pi_E = 1 - (0.7 - 2.2\pi_L) - 2.75\pi_L \rightarrow \pi_E = 0.3 - 0.55\pi_L$$

Passo 4: Encontrar o valor final de π_L

Substituímos o nosso conjunto de variáveis em função de π_L na Equação (2):

$$0.2(0.3 - 0.55\pi_L) - 0.6(0.7 - 2.2\pi_L) + 0.2\pi_L + 0.1(1.75\pi_L) = 0$$

$$(0.06 - 0.11\pi_L) - (0.42 - 1.32\pi_L) + 0.2\pi_L + 0.175\pi_L = 0$$

$$0.06 - 0.11\pi_L - 0.42 + 1.32\pi_L + 0.2\pi_L + 0.175\pi_L = 0$$

$$1.585\pi_L = 0.36$$

$$\pi_L = \frac{0.36}{1.585} \approx 0.2271$$

Passo 5: substituir de volta nas equações isoladas

$$\pi_A = 1.75(0.2271) \approx 0.3975$$

$$\pi_R = 0.7 - 2.2(0.2271) \approx 0.2003$$

$$\pi_E = 0.3 - 0.55(0.2271) \approx 0.1751$$

Logo, o vetor de probabilidades estacionárias da nossa Cadeia de Markov é:

$$\pi = [0.1751, \quad 0.2003, \quad 0.2271, \quad 0.3975]$$

Com isso, vemos que o estado **A** é que possui maior probabilidade estacionária.

Para ilustração, podemos ver o seguinte gráfico:

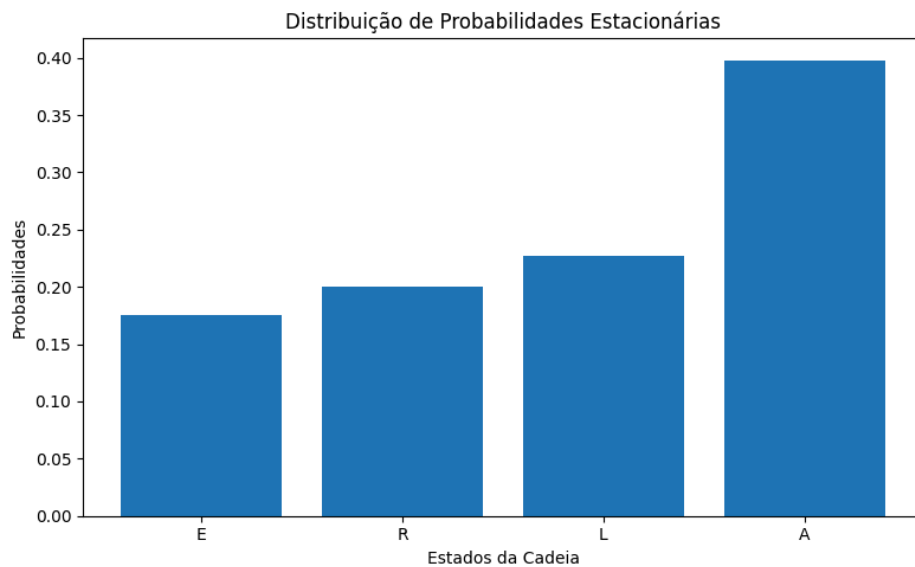


Figure 2: Vetor de probabilidades estacionárias

- v. Para avaliar o quão rápido o sistema “esquece” as probabilidades iniciais e converge para as estacionárias, iremos utilizar como base a Eq. 1

A fim de encontrar a velocidade de convergência da Cadeia de Markov em direção ao estado estacionário, foi determinado o tempo de “acomodação” do sistema por meio de um critério de tolerância, similar ao conceito de tempo de acomodação empregado em sistemas de controle. A estratégia consiste em estabelecer uma banda de erro predefinida (1%) em torno da probabilidade final esperada para cada estado e percorrer o histórico de iterações em ordem reversa para identificar o instante exato em que a curva ultrapassa esse limite de tolerância. O passo imediatamente posterior a essa saída define o ponto de estabilização do estado em questão.

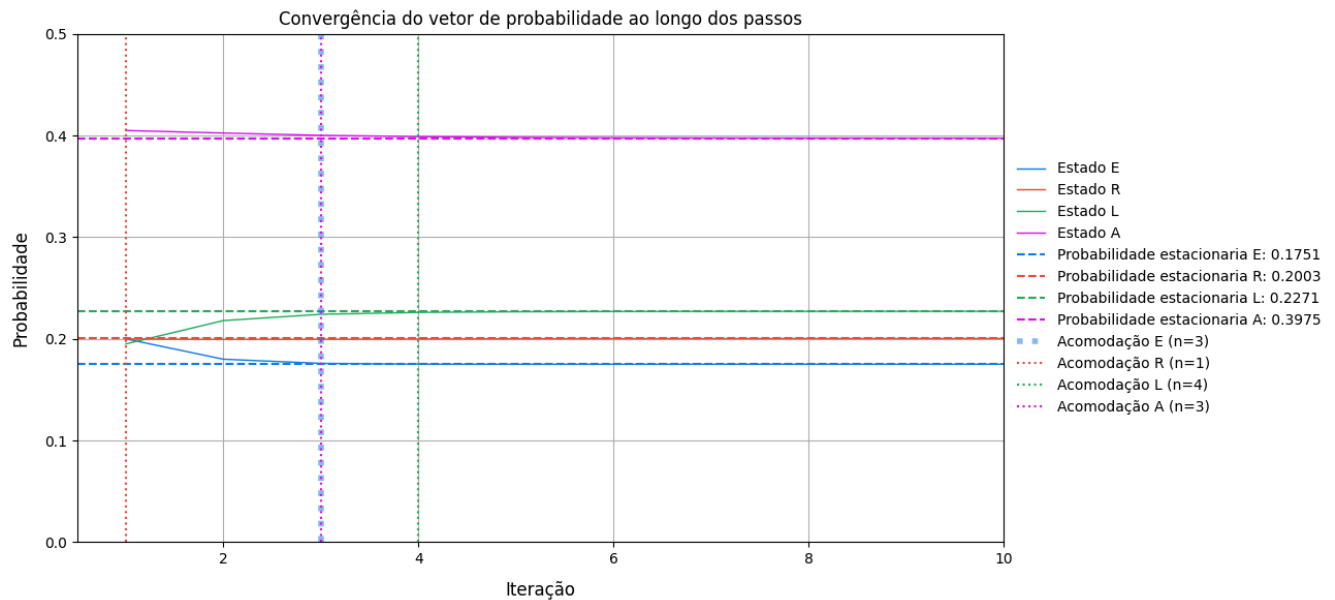


Figure 3: Plot de convergência do vetor de probabilidades

Podemos ver que:

- O estado “E” se aproxima da estacionária em $n = 3$
- O estado “R” em $n = 1$
- O estado “L” em $n = 4$
- O estado “A” em $n = 3$

Portanto, o sistema como um todo “esquece o estado inicial” a partir de $n = 4$, o limite superior dos “esquecimentos individuais”.

Códigos utilizados para resolução dos itens

```
import numpy as np
from matplotlib import pyplot as plt

# Definindo a matriz de transição de 1 passo
P = np.array([[0.3, 0.2, 0.25, 0.25],
              [0.3, 0.4, 0.15, 0.15],
              [0.1, 0.2, 0.5, 0.2],
              [0.1, 0.1, 0.1, 0.7]])

# Definindo uma lista para os valores desejados de "n"
ns = [1, 2, 5, 10, 20, 50, 100]

Ps = [np.linalg.matrix_power(P, n) for n in ns]
for key, item in zip(ns, Ps):
    print(f"Matriz de transição de {key} passos: \n {item}\n")

from typing import List, Tuple

def calcular_diffs(Ps: List[np.ndarray]) -> Tuple[List[np.ndarray], List[float]]:
    # Lista para armazenar as matrizes de diferenca
    diffs = []
    for i in range(1, len(Ps)):
        diff = Ps[i] - Ps[i-1]
        diffs.append(diff)

    # Lista para armazenar a soma do quadrado dos elementos
    # das matrizes de diferenca
    diffs_unitaria = [np.sum(mat**2) for mat in diffs]

    return diffs, diffs_unitaria

diffs, udiffs = calcular_diffs(Ps)

for prev, curr, diff in zip(ns[:-1], ns[1:], diffs):
    print(f'Matriz de diferença entre a matriz de {prev} passos e a de {curr} passos: \n {diff}\n')

for prev, curr, diff in zip(ns[:-1], ns[1:], udiffs):
    print(f'diferença "unitaria" entre a matriz de {prev} passos e a de {curr} passos: {diff}\n')

# Definindo os xs para utilizar no plot
xs = np.arange(1, len(udiffs) + 1)
plt.figure(figsize=(10, 6))

plt.plot(xs, udiffs,
         marker='o',
         markersize=5,
         linestyle='-',
         linewidth=1,
         color="#152361",
```



```

        alpha=1.0)

plt.title(r'Convergência: Soma dos Quadrados das Diferenças ( $P_i - P_{i-1}$ )',
        color="#000000")

plt.xlabel('Iteração ($i$)')
plt.ylabel('Soma dos Quadrados')

estados_iniciais = {
    'Estado E': np.array([1.0, 0.0, 0.0, 0.0]),
    'Estado R': np.array([0.0, 1.0, 0.0, 0.0]),
    'Estado L': np.array([0.0, 0.0, 1.0, 0.0]),
    'Estado A': np.array([0.0, 0.0, 0.0, 1.0])
}

for estado, p0 in estados_iniciais.items():
    print(f"Começando 100% no {estado}")

    print(f"n = {0:<3} | p(0) = [{p0[0]:.4f}, {p0[1]:.4f}, {p0[2]:.4f}, {p0[3]:.4f}]")
    for n in ns:
        # Eleva a matriz P a potência n e multiplica pelo vetor inicial
        Pn = np.linalg.matrix_power(P, n)
        pn = np.dot(p0, Pn)

        print(f"n = {n:<3} | p(n) = [{pn[0]:.4f}, {pn[1]:.4f}, {pn[2]:.4f}, {pn[3]:.4f}]")
        print("-" * 45)

# Plot do vetor estacionário
estados = ['E', 'R', 'L', 'A']
probabilidades = [0.1751, 0.2003, 0.2271, 0.3975]
plt.figure(figsize=(8, 5))

plt.bar(estados, probabilidades)

plt.title('Distribuição de Probabilidades Estacionárias')
plt.xlabel('Estados da Cadeia')
plt.ylabel('Probabilidades')

plt.tight_layout()
plt.show()

# Definindo o vetor de probabilidades iniciais descritos na questao
p0 = np.array([0.3, 0.2, 0.1, 0.4])
powers = np.arange(1, 50, 1)

# Definindo as matrizes de transicao de 1 a 100 passos
Ps = [np.linalg.matrix_power(P, power) for power in powers]
ps = [np.dot(p0, Pi) for Pi in Ps]

```

```

# Pegando as probabilidades individuais dos estados
Es = [p[0] for p in ps]
Rs = [p[1] for p in ps]
Ls = [p[2] for p in ps]
As = [p[3] for p in ps]

# Pegando o valor final de cada uma
Ey = Es[-1]
Ry = Rs[-1]
Ly = Ls[-1]
Ay = As[-1]
print(Ey, Ry, Ly, Ay)

def find_settling(probs: List, indexes: List, threshold: float = 0.01):
    final_value = probs[-1]
    upper = (1 + threshold) * final_value
    lower = (1 - threshold) * final_value

    for i in range(len(probs) - 1, -1, -1):
        if probs[i] < lower or probs[i] > upper:
            return indexes[i+1]

    return indexes[0]

settling_E = find_settling(Es, powers)
settling_R = find_settling(Rs, powers)
settling_L = find_settling(Ls, powers)
settling_A = find_settling(As, powers)

print(f"O estado E acomoda no passo: {settling_E}")
print(f"O estado R acomoda no passo: {settling_R}")
print(f"O estado L acomoda no passo: {settling_L}")
print(f"O estado A acomoda no passo: {settling_A}")

fig, ax = plt.subplots(figsize=(10, 6))

# Paleta de cores sóbria
cores = {'E': "#117de9", 'R': "#e74c3c", 'L': "#27ae60", 'A': "#dd12f3"}

# Plotando as linhas principais
ax.plot(powers[:10], Es[:10], label='Estado E', color=cores['E'], linewidth=1)
ax.plot(powers[:10], Rs[:10], label='Estado R', color=cores['R'], linewidth=1)
ax.plot(powers[:10], Ls[:10], label='Estado L', color=cores['L'], linewidth=1)
ax.plot(powers[:10], As[:10], label='Estado A', color=cores['A'], linewidth=1)

# Adicionando as assíntotas
ax.axhline(y=Ey, color=cores['E'], linestyle='--', linewidth=1.5, label =
f"Probabilidade estacionaria E: {Ey:.4f}")
ax.axhline(y=Ry, color=cores['R'], linestyle='--', linewidth=1.5, label =
f"Probabilidade estacionaria R: {Ry:.4f}")
ax.axhline(y=Ly, color=cores['L'], linestyle='--', linewidth=1.5, label =

```

```
f"Probabilidade estacionaria L: {Ly:.4f}")
ax.axhline(y=Ay, color=cores['A'], linestyle='--', linewidth=1.5, label =
f"Probabilidade estacionaria A: {Ay:.4f}")

ax.axvline(x=settling_E, color=cores['E'], linestyle=':', linewidth=4, alpha=0.5, label
= f"Acomodação E (n={settling_E})")
ax.axvline(x=settling_R, color=cores['R'], linestyle=':', label = f"Acomodação R
(n={settling_R})")
ax.axvline(x=settling_L, color=cores['L'], linestyle=':', label = f"Acomodação L
(n={settling_L})")
ax.axvline(x=settling_A, color=cores['A'], linestyle=':', alpha=1.0, label =
f"Acomodação A (n={settling_A})")

# Título e Eixos formatados
plt.title(r'Convergência do vetor de probabilidade ao longo dos passos',
color="#000000")

plt.xlabel("Iteração", fontsize=12, labelpad=10)
plt.ylabel("Probabilidade", fontsize=12, labelpad=10)

# Define os limites dos eixos
plt.xlim(0.5, len(powers[:10]))
plt.ylim(0, 0.5) # Ajuste para 0.5 para focar melhor nos dados

# Legenda fora do gráfico para não sobrepor as linhas
plt.tight_layout()
plt.grid(True)
plt.legend(loc='center left', bbox_to_anchor=(1, 0.5), frameon=False)

plt.show()
```